Representation of Distributed Polynomials

Declan Jagt, Carl Richardson

October 20, 2026

Abstract

This document describes the classes used in PIESOS to represent distributed polynomials, i.e. polynomials in distributed states $\mathbf{x} \in L_2^n[\Omega]$, and the functions used to pose conditions on such polynomials in a convex optimization program. The classes build on one another:

`sopvar`	PI operators (see the documentation of `sopvar`);
`intop`	PI functionals, acting on distributed monomials (Section 3.1);
`tensopvar`	sums of tensor products of PI operators (Section 3.2);
`tensopmat`	block matrices of `tensopvar` and `intop` objects (Section 3.3);
`polyopvar`	distributed polynomials (Section 4);
`piesos_*`	SOS programming functions (Section 5).

1 Preliminaries

Let $\mathbb{N}$ be the space of natural numbers, and $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$. For $d, N \in \mathbb{N}$, and $\Omega \subseteq \mathbb{R}^N$, let

$$\Omega^d := \underbrace{\Omega \times \cdots \times \Omega}_{d \text{ factors}},$$

and let $\Omega^0 = \emptyset$. Let $L_2^n[\Omega]$ denote the space of $\mathbb{R}^n$-valued square integrable functions, and define $L_2^n[\emptyset] := \mathbb{R}^n$. For vectors $x \in \mathbb{R}^m$ and $y \in \mathbb{R}^n$, define the Kronecker product in the standard way,

$$x \otimes y = \begin{bmatrix} x_1 y \\ \vdots \\ x_m y \end{bmatrix} \in \mathbb{R}^{mn}.$$

For $x, y \in \mathbb{R}^m$, define the Hadamard (elementwise) product

$$x \odot y = \begin{bmatrix} x_1 y_1 \\ \vdots \\ x_m y_m \end{bmatrix} \in \mathbb{R}^m.$$

For $\mathbf{x} \in L_2^m[\Omega_1]$ and $\mathbf{y} \in L_2^n[\Omega_2]$, define the tensor product (in the standard way)

$$(\mathbf{x} \otimes \mathbf{y})(s_1, s_2) = \mathbf{x}(s_1) \otimes \mathbf{y}(s_2) = \begin{bmatrix} \mathbf{x}_1(s_1)\mathbf{y}(s_2) \\ \vdots \\ \mathbf{x}_m(s_1)\mathbf{y}(s_2) \end{bmatrix}, \qquad (s_1, s_2) \in \Omega_1 \times \Omega_2,$$

where now $\mathbf{x} \otimes \mathbf{y} \in L_2^{mn}[\Omega_1 \times \Omega_2]$ (in fact, $L_2^m[\Omega_1] \otimes L_2^n[\Omega_2]$ is a dense subspace of $L_2^{mn}[\Omega_1 \times \Omega_2]$). Define the degree-$d$ distributed monomial in $\mathbf{x}$ as

$$\mathbf{x}^{\otimes d} = \underbrace{\mathbf{x} \otimes \cdots \otimes \mathbf{x}}_{d \text{ factors}}.$$

For linear operators $\mathcal{A} : L_2^n[\Gamma_1] \to L_2^m[\Omega_1]$ and $\mathcal{B} : L_2^q[\Gamma_2] \to L_2^p[\Omega_2]$, define the (proper) tensor product $\mathcal{A} \otimes \mathcal{B} : L_2^{nq}[\Gamma_1 \times \Gamma_2] \to L_2^{mp}[\Omega_1 \times \Omega_2]$ by

$$(\mathcal{A} \otimes \mathcal{B})(\mathbf{x} \otimes \mathbf{y})(s_1, s_2) = (\mathcal{A}\mathbf{x})(s_1) \otimes (\mathcal{B}\mathbf{y})(s_2), \qquad (s_1, s_2) \in \Omega_1 \times \Omega_2,$$

for $\mathbf{x} \in L_2^n[\Gamma_1]$ and $\mathbf{y} \in L_2^q[\Gamma_2]$. Now let $\mathcal{A} : L_2^n[\Gamma_1] \to L_2^m[\Omega_A]$ and $\mathcal{B} : L_2^q[\Gamma_2] \to L_2^m[\Omega_B]$ map to functions with the same vector dimension m, where Ω_A and Ω_B are products of intervals in the spatial variables on

which the outputs of $\mathcal{A}$ and $\mathcal{B}$ depend. Let Ω be the product of the intervals in all variables in either Ω_A or Ω_B, and for $s \in \Omega$, let s_A and s_B be the restrictions of s to the variables of Ω_A and Ω_B. We define the "pointwise" tensor product, $\mathcal{A} \oslash \mathcal{B} : L_2^{nq}[\Gamma_1 \times \Gamma_2] \to L_2^m[\Omega]$ by

$$(\mathcal{A} \oslash \mathcal{B})(\mathbf{x} \otimes \mathbf{y})(s) = (\mathcal{A}\mathbf{x})(s_A) \odot (\mathcal{B}\mathbf{y})(s_B), \qquad s \in \Omega,$$

for $\mathbf{x} \in L_2^n[\Gamma_1]$ and $\mathbf{y} \in L_2^q[\Gamma_2]$. If $\Omega_A = \Omega_B$, this is just the pointwise product of the outputs, whereas for variables on which only one of the outputs depends, the product is taken with a function that is constant in that variable. This operation corresponds to what we actually refer to as the tensor product of PI operators in the papers.

2 Representing Function Spaces for Distributed Monomials

The PIESOS programming framework is built on distributed monomials: for $\mathbf{x} \in L_2^n[\Omega]$, distributed polynomials are defined by coefficient operators acting on the distributed monomial basis, $\mathbf{Z}_d(\mathbf{x}) = (\mathbf{x}, \ldots, \mathbf{x}^{\otimes d})$, allowing global equality and positivity conditions (for all $\mathbf{x}$) to be posed as conditions on the coefficient operators. However, in order to numerically represent the necessary coefficient operators, we need a way of representing the function space they act on. In particular, we consider the generalized L_2 space of tensor products of states, defined as

$$L_2^n[[a_1, b_1]^{d_1} \times \cdots \times [a_N, b_N]^{d_N}]. \tag{1}$$

This function space can be represented by the vector dimension, n, the domains, $[a_i, b_i]$, and the degrees, d_i. Also storing the names of the variables, s_i, acting on the different domains, we represent this function space numerically by the following four arrays

- `dim`: scalar integer specifying the vector size of the state,
  `n`
- `vars`: $1 \times N$ `cellstr` specifying the names of the spatial variables,
  {`s1,...,sN`};
- `dom`: $N \times 2$ array specifying the intervals on which each of the spatial variables are defined,
  `[a1,b1; ...; aN,bN]`
- `deps`: $1 \times N$ array of integers, specifying for each variable to what order the state depends on it,
  `[d1, ..., dN]`

This will be the basic structure used by PIESOS to represent the tensor-product function space in (1). Note here that `deps(i)=0` can be used to indicate no dependence on variable `si`. Using this structure, for an operator

$$\mathcal{A} : L_2^m[[a_1, b_1]^{p_1} \times \cdots \times [a_N, b_N]^{p_N}] \to L_2^n[[c_1, d_1]^{q_1} \times \cdots \times [c_M, d_M]^{q_M}]$$

we will represent this input and output domain of the operator as a structs,

- `dims`: `struct` with fields
  `in=m` and `out=n`;
- `vars`: `struct` with fields
  `in={t1,...,tN}` and `out={s1,...,sM}`;
- `dom`: `struct` with fields
  `in=[a1,b1; ...; aN,bN]` and `out=[c1,d1; ...; cM,dM]`;
- `deps`: `struct` with fields
  `in=[p1, ..., pN]` and `out=[q1, ..., qM]`;

Operators that are defined by several factors, blocks or states (`tensopvar`, `tensopmat`, `polyopvar`) store the degrees of each factor, block or state in a matrix `depmat`, with one row for each, and `deps` is computed from this matrix.

Notes:

- The input and output variables may overlap. The same variable in the input and output must have the same domain: if `vars.in(i)=vars.out(j)` then `dom.in(i,:)=dom.out(j,:)`.
- The variable names in `vars.in` and `vars.out` are always stored in sorted order. The other fields (`dom`, `deps`, `depmat`, and the parameters of the operator) follow this order, so that e.g. `dom.in(i,:)` is the interval of `vars.in{i}`.
- For a state $\mathbf{x} \in L_2^n[\Omega]$ in N variables, $s = (s_1, \dots, s_N) \in \Omega$, the dth-order tensor product $\mathbf{x}^{\otimes d} \in L_2^{n^d}[\Omega^d]$ will depend on all variables to the same order d: `deps=[d,...,d]`. Although this will be the most common case, however, we may also encounter coupled systems with different states depending on different variables, e.g. $\mathbf{x}(s_1, s_2)$ and $\mathbf{y}(s_2)$. The vector-valued `deps` is necessary to represent such mixed products as well, e.g. $\mathbf{x} \otimes \mathbf{y}$ will have `deps=[1,2]`.
- Perhaps it would make more sense to use `doms` instead of `dom`, given that we also have `vars` and `dims`, but the current implementation uses `dom` to match the `sopvar` structure.

In the following section, we will use this structure to propose a numerical representation of operators between function spaces as in (1).

3 Classes of Operators on Distributed Monomials

To represent distributed polynomials, we will use two main classes of operators: PI functionals (or F-PI operators), mapping distributed monomials to $\mathbb{R}^m$, and tensor-PI operators, representing tensor products of PI operators, mapping between different function spaces. We represent these operators by `intop` and `tensopvar` objects, described in Subsections 3.1 and 3.2, respectively. In order to represent maps between coupled function spaces, we will use the `tensopmat` structure described in Subsection 3.3, which allows for concatenation of `intop` and `tensopvar` objects.

3.1 The class of PI Functionals: `intop`

The `intop` class represents a class of functionals mapping distributed monomials to finite-dimensional vectors,

$$\mathcal{K} : L_2^n[[a_1, b_1]^{d_1} \times \cdots \times [a_N, b_N]^{d_N}] \to \mathbb{R}^m$$

Here, in order to represent a broad class of functionals, including involving nested integrals, $\int_a^b \int_a^x \mathbf{w}(x, y)\, dydx$, we define the following indicator functions,

$$\boldsymbol{\chi}_{[x,y]}(z) = \begin{cases} 1, & x < y,\ z \in [x, y], \\ -1, & x > y,\ z \in [y, x], \\ 0, & x \neq y,\ z \notin [\min(x, y), \max(x, y)], \\ \boldsymbol{\delta}(z - x), & x = y, \end{cases} \qquad x, y, z \in \mathbb{R},$$

where $\boldsymbol{\delta}$ is the Dirac delta, so that, e.g.

$$\int_{[a,b]^2} \boldsymbol{\chi}_{[a,x]}(y)\mathbf{w}(x, y)\, dy\, dx = \int_a^b \int_a^x \mathbf{w}(x, y)\, dy\, dx, \qquad \int_{[a,b]^2} \boldsymbol{\chi}_{[x,x]}(y)\mathbf{w}(x, y)\, dy\, dx = \int_a^b \mathbf{w}(x, x)\, dx,$$

assuming sufficient regularity of $\mathbf{w}$. Note that for $x > y$, the indicator function is negative, so that $\int \boldsymbol{\chi}_{[x,y]}(z)\, dz = -\int_y^x dz$, i.e. integrals over reversed limits are signed integrals. This is the convention implemented in `apply_op`, which evaluates the integrals one variable at a time as $\int_L^U$, with $L > U$ allowed. E.g. for $s_1, s_3 \in [0, 1]$ and $s_2 \in [s_1, s_3]$, $\iiint_{s_1}^{s_3} s_1\, ds_2 ds_3 ds_1 = -\frac{1}{12}$, whereas if the indicator were zero for $x > y$ we would get $\frac{1}{24}$. For $x_1, \dots, x_d \in [a, b]$ and given (row) vectors $\alpha, \beta \in \{0, \dots, d + 1\}^d$, then, we let $x_0 := a$ and $x_{d+1} := b$ and define the indicator function

$$\mathbb{I}_{\alpha,\beta}(x) = \boldsymbol{\chi}_{[x_{\alpha_1}, x_{\beta_1}]}(x_1) \cdots \boldsymbol{\chi}_{[x_{\alpha_d}, x_{\beta_d}]}(x_d).$$

This allows a broad class of different nested integrals in variables $x_1, \dots, x_d \in [a, b]$ to be represented. For the integral to be well-defined, we require that $\alpha_j \neq d + 1$ and $\beta_j \neq 0$ for all j (the lower limit cannot be the upper boundary and vice versa), that at least one variable is integrated over the full interval, $\alpha_j = 0$

and $\beta_j = d+1$ for some j, and that the limits are not mutually inconsistent (e.g. circular). Given, domains $[a_i, b_i] \subseteq \mathbb{R}$, degrees $d_i \in \mathbb{N}_0$, and matrices $A_i, B_i \in \{0, \ldots, d_i+1\}^{q_i \times d_i}$, we can define a functional

$$\mathcal{K} : L_2^n[[a_1, b_1]^{d_1} \times \cdots \times [a_N, b_N]^{d_N}] \to \mathbb{R}^m$$

by parameters $K_j(s) \in \mathbb{R}^{m \times n}[s]$ as

$$\mathcal{K}\mathbf{w} = \sum_{j_1=1}^{q_1} \cdots \sum_{j_N=1}^{q_N} \int_{[a_1,b_1]^{d_1}} \cdots \int_{[a_N,b_N]^{d_N}} \mathbb{I}_{A_1(j_1,:),B_1(j_1,:)}(s_1) \cdots \mathbb{I}_{A_N(j_N,:),B_N(j_N,:)}(s_N) \boldsymbol{K}_j(s_1, \ldots, s_N)\mathbf{w}(s_1, \ldots, s_N)\, ds_N \cdots ds_1 \tag{2}$$

where $j = (j_1, \ldots, j_N)$, $s_i = (s_{i,1}, \ldots, s_{i,d_i})$ for $i \in \{1, \ldots, N\}$, and where we let $A(k,:)$ denote the kth row of matrix A. Here, each K_j is polynomial, and may be defined by a vector of decision variables. We express all the parameters K_j in terms of the same decision variables and basis of monomials, so that

$$K_j(s_1, \ldots, s_N) = \left(I_m \otimes \begin{bmatrix}1\\ \xi\end{bmatrix}\right)^T C_j(I_n \otimes Z_1(s_1) \otimes \cdots \otimes Z_N(s_N)), \qquad j = (j_1, \ldots, j_N),\ j_i \in \{1, \ldots, q_i\}, \tag{3}$$

where $\xi \in \mathbb{R}^{n_d}$ are the decision variables, $Z_i(s_i) \in \mathbb{R}^{n_{Z_i}}$ are monomial vectors in the variables $s_{i,1}, \ldots, s_{i,d_i}$, and $C_j \in \mathbb{R}^{m(n_d+1) \times nn_Z}$ for $n_Z = n_{Z_1} \cdots n_{Z_N}$. In other words, the rows of C_j are ordered with the matrix row (m) outer and the decision variable inner (constant term first), and the columns with the matrix column (n) outer and the monomial inner, the same format as the coefficients of a `dpvar` object. We represent such a functional by an `intop` object, through the following fields.

intop structure:

- `params`: $q_1 \times \cdots \times q_N$ cell, with each element a $m(n_d+1) \times nn_Z$ `sparse` array. Element $j = (j_1, \ldots, j_N)$ represents the coefficient matrix C_j defining the parameters $K_j(s)$. Note that these are the coefficients, not the kernels K_j themselves, which are returned by `kernels(Kop)`;
- `Zd`: $n_d \times 1$ `cellstr` specifying the names of the decision variables, ξ. A parameter that does not depend on all decision variables has zero rows for those it does not use;
- `Z`: `struct` with fields `in` and `out`, representing the monomial basis in terms of which the parameters are defined. Specifically, `Z.in` is a $1 \times N$ cell, with each element a $n_{Z_i} \times d_i$ matrix of integers specifying the degrees of the dummy variables defining monomial vector Z_i as per `polynomial` format, where the rows must be distinct. Then $n_Z = n_{Z_1} \cdots n_{Z_N}$, where the monomials of the first variable change slowest. The field `Z.out` will be a 1×0 cell, as there are no "output" monomials;
- `intmat`: $1 \times N$ cell, with each element a $q_i \times 2d_i$ matrix specifying the indicator functions for the dummy variables associated with variable s_i (see the first note below: these are not necessarily the intervals of integration). Each row of `intmat{i}` corresponds to a different term along the ith direction (multiplied with different terms along the other directions), and columns $(2j-1, 2j)$ for $j \in \{1, \ldots, d_i\}$ specify the indicator function of variable $s_{i,j}$ in that term. Values range over $\{0, \ldots, d_i+1\}$, with `intmat{i}(k,[2j-1,2j])=[l,u]` indicating an indicator function $\boldsymbol{\chi}_{[s_{i,l}, s_{i,u}]}(s_{i,j})$. Here, we set $s_{i,0} := a_i$ and $s_{i,d_i+1} := b_i$, and $s_{i,l} = s_{i,u}$ is used to represent a Dirac delta, so that the term acts as a multiplier along $s_{i,j}$, evaluating $\mathbf{w}$ at $s_{i,j} = s_{i,l}$. Note that in (2), `intmat{i}(:,1:2:end)` corresponds to matrix A_i, and `intmat{i}(:,2:2:end)` corresponds to matrix B_i. The conditions on A_i, B_i stated above are checked when declaring an object, and boundary evaluations (`[0,0]` or `[d+1,d+1]`) are not supported;
- `vars`: struct with fields `in` and `out`, which are $1 \times N$ and $1 \times M$ `cellstr` objects specifying the input and output variable names of the operator, respectively. Since `intop` object represent functionals, we will have $M = 0$;
- `dom`: struct with fields `in` and `out`, which are $N \times 2$ and $M \times 2$ numeric arrays specifying the intervals on which the input and output variables are defined. Again, we will have $M = 0$;

The following properties are computed from these fields, and cannot be set.

- `deps` (or `depmat`): struct with fields `in` and `out`, specifying the degree to which the input and output of the operator depends on each of the input and output variables. In particular, we will have `deps.in=[d1,...,dN]`, determined from the dimensions of `intmat`, and `deps.out=zeros(1,0)`;

- `matdim`: 1×2 array of integers, specifying the matrix dimensions of the operator, $[m, n]$. The values are determined by the dimensions of the coefficients in `params`, together with `Zd` and `Z`: $m =$ `size(params{1},1)/(nd+1)` and $n =$ `size(params{1},2)/nZ`. Also `dims` is available, as a struct with `dims.in`$= n$ and `dims.out`$= m$.

Construction: An `intop` object can be declared in three ways:

- `Kop = intop(params,intmat,vars,dom,Z,Zd)`, from the sparse coefficients, with `Z` the struct (or the cell `Z.in`), and `Zd` optional, so that the arguments directly match the fields;
- `Kop = intop(kernels,intmat,vars,dom)`, from a $q_1 \times \cdots \times q_N$ cell `kernels` of $m \times n$ objects of type `double`, `polynomial` or `dpvar`. The kernels may only depend on the dummy variables of the functional, and are expressed in terms of the union of all monomials and decision variables that appear in them;
- `Kop = intop(K)`, from a constant K of type `double`, `polynomial` or `dpvar`, representing a functional with $N = 0$ and no integrals.

Examples:

- The functional $\mathcal{K}\mathbf{w} = \int_0^1 \mathbf{w}(s,s)\,ds$ acts on $L_2[[0,1]^2]$ (a state of degree 2 in s), with $N = 1$, $d_1 = 2$, and `intmat{1}=[0,3,1,1]`: s_1 has the indicator function $\boldsymbol{\chi}_{[a,b]} = \boldsymbol{\chi}_{[s_0,s_3]}$, and s_2 the delta $\boldsymbol{\chi}_{[s_1,s_1]}$. Taking $\mathbf{w} = \mathbf{u} \otimes \mathbf{u}$, this is the functional $\langle \mathbf{u}, \mathbf{u} \rangle$.
- The functional $\int_0^1 \int_0^{s_1} (1 + s_1 s_2)\mathbf{w}(s_1, s_2)\,ds_2 ds_1$ is defined by `intmat{1}=[0,3,0,1]`, and the kernel $1 + s_1 s_2$, with $Z_1 = [1;\, s_1 s_2]$ so that `Z.in{1}=[0,0;1,1]` and `params{1}=[1,1]`.
- With decision variables $\xi = (\xi_1, \xi_2)$, the kernel $K(s) = \xi_1 + 2\xi_2 s_1$ is expressed in terms of $Z_1 = [1;\, s_1]$ and $[1; \xi_1; \xi_2]$ by

$$C = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad K(s) = \begin{bmatrix} 1 & \xi_1 & \xi_2 \end{bmatrix} C \begin{bmatrix} 1 \\ s_1 \end{bmatrix}.$$

Notes:

- The matrices `intmat` define the product of indicator functions $\mathbb{I}_{A_i,B_i}$ in (2), and thus constraints on the dummy variables, not directly the interval over which each variable is integrated. In particular, `intmat{i}(k,[2j-1,2j])=[l,u]` does not necessarily mean that $s_{i,j}$ ranges over $[s_{i,l}, s_{i,u}]$, as its range also depends on the indicator functions of the other variables. For $d_i = 2$, e.g.,

$$\texttt{intmat\{i\}=[0,3,0,1]}: \quad \boldsymbol{\chi}_{[a,b]}(s_1)\boldsymbol{\chi}_{[a,s_1]}(s_2) \quad \Rightarrow \quad \int_a^b \int_a^{s_1} \cdots\, ds_2 ds_1,$$

  where the indicator function of s_1 is $\boldsymbol{\chi}_{[a,b]}$, but, as $s_2 \le s_1$, it is the other indicator function that makes s_1 range over $[s_2, b]$ for fixed s_2. The same functional is therefore defined by `intmat{i}=[2,3,0,3]`, $\boldsymbol{\chi}_{[s_2,b]}(s_1)\boldsymbol{\chi}_{[a,b]}(s_2)$, where now s_2 has the full domain (tested: both give 1/10 for $\mathbf{w} = s_1^2 s_2$ on $[0,1]$). Similarly, `[0,3,1,1]` contains $\boldsymbol{\chi}_{[s_1,s_1]}(s_2) = \boldsymbol{\delta}(s_2 - s_1)$, so that s_2 is not integrated over any interval at all, but set equal to s_1. In the code, the integrals are evaluated one variable at a time, the innermost first (see `int_order` below), where the limits of a variable may involve variables that are integrated later; the result is the integral of the product of the indicator functions, with the signed convention above if the limits are reversed.
- For given input variables, the dummy variables used for integration are defined by appending `_dum` to the variable names, along with an index ranging from 1 up to the order of the variable. For example, if `vars.in={s1,s2,s3}` and `deps.in=[2,1,2]`, then the full list of dummy variables will be {`s1_dum1,s1_dum2,s2_dum1,s3_dum1,s3_dum2`}. These variables can be extracted as `polynomial` class objects using `Kop.pvars`, and their names using `Kop.varnames`.
- The order in which the integrals must be taken follows from `intmat`, and is returned by `Kop.int_order`: `int_order{i}(k,j)=q` implies that variable $s_{i,j}$ in term k must be integrated qth, with the inner integral first. For `intmat{1}=[0,3,0,1]`, s_2 is integrated first, and s_1 second.
- Most of the `out` fields of `intop` objects will be empty, since `intop` objects are used to represent functionals, with the output involving no spatial variables. Nevertheless, the `in/out` structure is nevertheless used for symmetry with respect to the other PIESOS classes, allowing for easier interface with the `tensopmat` class.

- The parameters are stored as sparse arrays. Since the number of rows scales with the number of decision variables, n_d, which may be very large, no dense arrays with n_d rows are ever formed by the methods of the class.

3.1.1 Addition of `intop` objects

Two functionals can only be added if they act on the same function space: the same variables, degrees `deps` and domains, and the same dimensions. Both are first expressed in terms of the same monomials `Z.in` and decision variables `Zd`, and in terms of the same set of integrals (`common_intmat`): an integral that appears in only one of the functionals is added to the other with zero coefficients. The sum is then given by the sum of the (sparse) coefficient matrices `params`, so that the cost does not depend on the number of decision variables beyond the number of nonzeros. Subtraction and negation follow directly. Terms that involve the same integral are combined by `unique_intmat`, and terms that are zero are removed by `clean_op`.

3.1.2 Multiplication of `intop` objects

Since `intop` objects represent maps from function spaces to $\mathbb{R}^m$, composition of two `intop` objects is not supported. The following products are supported.

- Left-multiplication by a multiplier $M \in \mathbb{R}^{p\times m}$ of type `double`, `polynomial` or `dpvar`, representing the functional $\mathcal{K}' = M\mathcal{K} : L_2^n[\cdots] \to \mathbb{R}^p$. For constant M, this acts directly on the coefficients `params`. For polynomial or `dpvar` M, the kernels are multiplied by M and then expressed in terms of the same monomials and decision variables again. The elementwise product, `M.*Kop`, is defined likewise.
- Composition with a tensor-PI operator on the right, $\mathcal{K}\mathcal{R}$, for $\mathcal{R}$ a `tensopvar` object (see Subsection 3.2). This represents the functional $\mathbf{w} \mapsto \mathcal{K}(\mathcal{R}\mathbf{w})$ acting on the input space of $\mathcal{R}$, again by an `intop` object. If $\mathcal{R}$ is a `tensopmat` with a single row, the result is the `tensopmat` of the compositions of $\mathcal{K}$ with each of the elements.

Tensor products are supported, where we distinguish between pointwise tensor products and proper tensor products. For $\mathcal{K}_A : L_2^{n_1}[\Gamma_A] \to \mathbb{R}^{m_1}$ and $\mathcal{K}_B : L_2^{n_2}[\Gamma_B] \to \mathbb{R}^{m_2}$, `otimes(A,B,type)` returns the functional $\mathcal{K}_C$ acting on $L_2^{n_1 n_2}[\Gamma_A \times \Gamma_B]$ that is defined by

$$\mathcal{K}_C(\mathbf{x}\otimes\mathbf{y}) = \begin{cases} (\mathcal{K}_A\mathbf{x}) \odot (\mathcal{K}_B\mathbf{y}), & \texttt{type(1)=1},\ m_1 = m_2 \text{ (or one of them is 1)}, \\ (\mathcal{K}_A\mathbf{x}) \otimes (\mathcal{K}_B\mathbf{y}), & \texttt{type(1)=2}. \end{cases}$$

The default is the pointwise product, `type` $= [1, 2]$, and only a proper tensor product along the input, `type(2)=2`, is supported. If $\mathcal{K}_A$ and $\mathcal{K}_B$ depend on the same spatial variable s_i, then this is the same variable, with the same domain, in both: the functional $\mathcal{K}_C$ integrates over the dummy variables of both, where the dummy variables of $\mathcal{K}_B$ are numbered after those of $\mathcal{K}_A$, so that $\mathcal{K}_C$ is of degree $d_i^A + d_i^B$ in s_i. A constant (a kernel that does not depend on the dummy variables) is treated as a functional without integrals. This is how the product of two distributed polynomial functionals is represented, e.g. $\int_a^b x(s)ds \cdot \int_a^b y(s)ds$ acts on $\mathbf{x}\otimes\mathbf{y} \in L_2[[a,b]^2]$.

3.1.3 Evaluation of `intop` objects

For a function $\mathbf{v}$ of type `polynomial` in the spatial variables `Kop.vars.in` (and possibly other variables), `apply_op(Kop,v)` returns the $m \times p$ `polynomial` (or `dpvar`, if `Kop` has decision variables) $\mathcal{K}\mathbf{v}$. Here, the integrals of the products of the monomials in the kernels and $\mathbf{v}$ are computed one spatial variable at a time (the limits of integration along a variable only involve that variable), after which the result is contracted with the sparse coefficients `params`. The integrals do not depend on the decision variables.

3.1.4 Canonical form

Different parameters may define the same functional: e.g. $\int_a^b \int_a^b K(s_1, s_2)\mathbf{w}(s_1, s_2)ds_2 ds_1$ can be written as the sum of the integrals over $s_2 \le s_1$ and $s_2 \ge s_1$, and the copies of $\mathbf{w} = \mathbf{x}\otimes\mathbf{x}$ may be exchanged. The method `combine_terms` brings the functional to a canonical form, writing each integral as a sum of integrals over ordered cells (every variable integrated from a up to the next larger variable, the largest over $[a, b]$) and, if the monomial on which the functional acts is specified (`combine_terms(Kop,degs,dims,depmat)`), ordering the copies of the same state, so that functionals that are equal have equal `intmat` and kernels, and their coefficients can be compared coefficientwise (see `piesos_eq`). The methods `reorder_states` and `reorder_vars` permute the factors of the distributed monomial on which the functional acts.

3.2 The class of Tensor-PI Operators: `tensopvar`

The PI functionals represented by `intop` objects allow for representation of distributed polynomial functionals, where the output is a finite-dimensional vector. In order to represent distributed polynomial functions, mapping to more general function spaces, we use the class of tensor-PI operators. Here, for domains $\Omega_i \subseteq \mathbb{R}^{N_i}$ for $i \in \{1,2,3\}$, a PI operator $\mathcal{P} : L_2^m[\Omega_1 \times \Omega_3] \to L_2^n[\Omega_2 \times \Omega_3]$ takes the form

$$(\mathcal{P}\mathbf{x})(s_2,s_3) = \sum_{\alpha\in\{-1,0,1\}^{N_3}} \int_{\Omega_1}\int_{\Omega_3} \mathbf{I}_\alpha(s_3,\theta_3)\boldsymbol{P}_\alpha(s_2,s_3,\theta_1,\theta_3)\mathbf{x}(\theta_1,\theta_3)\,d\theta_3\,d\theta_1$$

for polynomial parameters $\boldsymbol{P}_\alpha$, and where

$$\mathbf{I}_\alpha(s_3,\theta_3) = \prod_{i=1}^{N_3} \mathbf{I}_{\alpha_i}(s_{3,i},\theta_{3,i}), \qquad \mathbf{I}_j(x,y) = \begin{cases} \mathbf{1}_{x\le y}, & j=-1,\\ \boldsymbol{\delta}(x-y), & j=0,\\ \mathbf{1}_{x\ge y}, & j=1, \end{cases}$$

for $\boldsymbol{\delta}$ the Dirac delta. Such 3-PI operators can be represented as `sopvar` objects, and we refer to the associated documentation for more information. The class of `tensopvar` objects allows for representation of tensor products of such PI operators. Recall, here, that we define two types of such tensor products: for linear operators $\mathcal{A} : L_2^n[\Gamma_1] \to L_2^m[\Omega_1]$ and $\mathcal{B} : L_2^q[\Gamma_2] \to L_2^p[\Omega_2]$, define the (proper) tensor product

$$(\mathcal{A}\otimes\mathcal{B})(\mathbf{x}\otimes\mathbf{y})(s_1,s_2) = (\mathcal{A}\mathbf{x})(s_1)\otimes(\mathcal{B}\mathbf{y})(s_2), \qquad (s_1,s_2)\in\Omega_1\times\Omega_2,$$

and for $\mathcal{A} : L_2^n[\Gamma_1] \to L_2^m[\Omega_A]$ and $\mathcal{B} : L_2^q[\Gamma_2] \to L_2^m[\Omega_B]$ (mapping to functions of the same vector dimension), we define the "pointwise" tensor product,

$$(\mathcal{A}\oslash\mathcal{B})(\mathbf{x}\otimes\mathbf{y})(s) = (\mathcal{A}\mathbf{x})(s_A)\odot(\mathcal{B}\mathbf{y})(s_B), \qquad s\in\Omega,$$

see Section 1 for the notation. We will refer to this pointwise tensor product as a tensor product of type 1, and the proper tensor product as one of type 2, for numerical representation. The `tensopvar` class supports representation of both types of tensor products. In particular, for PI operators

$$\mathcal{P}_{i,j} : L_2^{n_j}[[a_1,b_1]^{q_{1,j}}\times\cdots\times[a_N,b_N]^{q_{N,j}}] \to L_2^{m_j}[[c_1,d_1]^{p_{1,j}}\times\cdots\times[c_M,d_M]^{p_{M,j}}]$$

for $i\in\{1,\dots,n_{\text{terms}}\}$ and $j\in\{1,\dots,n_{\text{factors}}\}$ and where $p_{k,j},q_{k,j}\in\{0,1\}$, we consider tensor-PI operators of type 2,

$$\mathcal{R} := \sum_{i=1}^{n_{\text{terms}}} \mathcal{P}_{i,1}\otimes\cdots\otimes\mathcal{P}_{i,n_{\text{factors}}}.$$

Furthermore, if the output dimensions of the operators match, i.e. $m_1=\cdots=m_{n_{\text{factors}}}$, then we may also represent the tensor-PI operators of type 1,

$$\mathcal{R} := \sum_{i=1}^{n_{\text{terms}}} \mathcal{P}_{i,1}\oslash\cdots\oslash\mathcal{P}_{i,n_{\text{factors}}}.$$

In both cases, the same factor j must map between the same function spaces in each of the terms i. Both types of tensor-PI operators can be represented numerically by `tensopvar` objects, defined by the following fields.

tensopvar structure:

- `ops`: $n_{\text{terms}}\times n_{\text{factors}}$ cell, storing the different operator $\mathcal{P}_{i,j}$ defining the tensor-PI operators as `sopvar` objects. Elements may also be `intop` objects (for a factor acting on a distributed monomial and mapping to $\mathbb{R}^m$), or `tensopvar` objects (nested operators, see the Notes). Note that for any fixed $j\in\{1,\dots,n_{\text{factors}}\}$, the operators $\mathcal{P}_{i,j}$ must map between the same input and output function spaces for all $i\in\{1,\dots,n_{\text{terms}}\}$;
- `otype`: 1×2 array specifying what type of tensor product is taken along both the input (second element) and output (first element) dimension. Here `otype=[1,2]` represents a pointwise tensor product, $\oslash$, and `otype=[2,2]` represents a proper (Kronecker) tensor product, $\otimes$. Although the value of `otype(2)` could be set to 1, this would represent an operator acting on pointwise products of states, $\mathbf{x}\oslash\mathbf{y}$. This functionality is included in order to support a possible adjoint of tensor-PI operators (not yet implemented), but in practice, almost only `otype(2)=2` will be encountered (`apply_op` does not support `otype(2)=1`).

The following properties are determined from the operators in `ops` (by the method `initialize`), and are protected: they cannot be set.

- `dims`: $n_{\texttt{factors}} \times 2$ array with each row j specifying the row (first column) and column (second) dimensions, $[m_j, n_j]$, of the operators $\mathcal{P}_{i,j}$ in factor j (recall that they must have the same dimensions in each term i). `dims.in` and `dims.out` return the second and first column, respectively.
- `vars`: struct with fields `in` and `out`, specifying the names of the spatial variables on which the input and output of the tensor-PI operator depend. These values are determined by unifying and sorting the input and output variables of the operators in `ops{i,j}.vars`.
- `dom`: struct with fields `in` and `out`, specifying the intervals on which the input and output variables are defined. These values are determined by unifying and sorting the input and output variables of the operators in `ops{i,j}.dom`. Of course, the same variable in different PI operators in `ops` must also have the same domain.
- `depmat`: struct with field `in` and `out`, containing $n_{\text{factors}} \times N$ and $n_{\text{factors}} \times M$ arrays of integers, respectively. Element (j, k) of these arrays specifies to what degree the input/output of the operators in factor j depends on the kth input/output variable specified in `vars`: 1 if the operators map from/to a function of this variable, and 0 otherwise. Values larger than 1 only occur for nested operators. This field is determined by comparing the variables in each factor, `ops{:,j}.vars`, to those of the full operator, `vars`.
- `matdim`: 1×2 array specifying the full matrix dimensions of the tensor-PI operator. If `otype(1)=1`, this is computed as `matdim(1)=dims(1,1)=...=dims(end,1)`. If `otype(1)=2`, this is computed as `matdim(1)=dims(1,1)*...*dims(end,1)`. Similarly for `otype(2)` and the column dimension.
- `deps`: struct with fields `in` and `out`, determining the degree to which the inputs and outputs of the full tensor-PI operator depend on the different variables. If `otype(1)=1`, this is determined as `max(depmat.out,[],1)`, otherwise, it is determined as `sum(depmat.out,1)`. Same for `otype(2)` and the input `depmat`.

Notes:

- `otype(1)=1` is only supported if the output dimensions of all the factors in all the terms of the tensor-PI operator match: `dims(1,1)=...=dims(end,1)`. The operators may map to functions on different domains, but products along the same variable will be taken pointwise, so that e.g. $(\mathcal{P}_1\mathbf{x})(s_1, s_2)$ and $(\mathcal{P}_2\mathbf{y})(s_2)$ implies $(\mathcal{P}_1 \oslash \mathcal{P}_2)(\mathbf{x} \otimes \mathbf{y})(s_1, s_2) = (\mathcal{P}_1\mathbf{x})(s_1, s_2)(\mathcal{P}_2\mathbf{y})(s_2)$.
- The `tensopvar` structure also supports nested operators, where each element of `ops` is itself a `tensopvar` object, or perhaps a `intop` object. This is occasionally useful for representing different combinations of tensor products, e.g. $\mathcal{P}_1 \otimes (\mathcal{P}_2 \oslash \mathcal{P}_3) \otimes \mathcal{P}_4$. In this case, the associated rows in `depmat.in` and `depmat.out` may involve values greater than 1, corresponding to the values of `deps.in` and `deps.out` of the `tensopvar` object in the factor. The number of factors including those of nested operators is returned by `num_factors`. A nested operator also arises when adding operators of different structure, see below.
- For a single factor, there is no distinction between a pointwise and a proper tensor product, and `otype` just records the type that was requested.

Example: Let `T1` be the `tensopvar` object representing the identity operator on $L_2[0, 1]$ (in the variable s). Then `otimes(T1,T1)` is the type-$[1, 2]$ product with `ops` of size 1×2, `deps.in=2` and `deps.out=1`: it maps $\mathbf{x} \otimes \mathbf{y} \in L_2[[0, 1]^2]$ to the function $s \mapsto x(s)y(s)$. By contrast, `otimes(T1,T1,[2,2])` has `deps.out=2`, mapping to the function $(s_1, s_2) \mapsto x(s_1)y(s_2)$.

3.2.1 Addition of `tensopvar` objects

Two tensor-PI operators can only be added if they have the same matrix dimensions, the same variables and domains, and the same degrees `deps`. The sum is then represented in one of the following ways.

- If both operators consist of a single factor, the factors are added (e.g., as `sopvar` objects).
- If both have the same `dims`, `depmat` and `otype`, their terms are concatenated: the rows of `ops` are stacked.
- Otherwise, the two operators are retained as the two terms of a nested operator with a single factor: `ops={A;B}`, with `otype=[2,2]`.

3.2.2 Multiplication of `tensopvar` objects

A tensor-PI operator can be multiplied from the left by a multiplier, or composed with another tensor-PI operator. For a matrix $M \in \mathbb{R}^{m\times p}$ of type `double` or `dpvar` (or a `polynomial` that is the identity), the operator $M\mathcal{R}$ is represented by multiplying the first factor of each term by M. This is only correct for a single factor, or a scalar M, where scalar multiplication commutes with the tensor product; for a general matrix and several factors, a Kronecker structure, $M \otimes I \otimes \cdots$ would be needed, which is not supported. For the composition of two tensor-PI operators $\mathcal{AB}$, with an equal number of factors and a proper tensor product in the output of $\mathcal{B}$ and the input of $\mathcal{A}$, `A.otype(2)=B.otype(1)=2`, we use the mixed product property

$$(\mathcal{A}_1 \otimes \cdots \otimes \mathcal{A}_d)(\mathcal{B}_1 \otimes \cdots \otimes \mathcal{B}_d) = \mathcal{A}_1\mathcal{B}_1 \otimes \cdots \otimes \mathcal{A}_d\mathcal{B}_d,$$

so that each pair of terms of $\mathcal{A}$ and $\mathcal{B}$ gives a term of the product, with the factors composed one by one. The output function space of $\mathcal{B}$ must match the input space of $\mathcal{A}$. If $\mathcal{A}$ is a nested operator, and $\mathcal{B}$ has a factor for each of the factors of $\mathcal{A}$ counted by `num_factors`, the factors of $\mathcal{B}$ are distributed over those of $\mathcal{A}$, and composed recursively. Composition of operators involving pointwise products of the outputs/inputs of multiple factors is not supported. Multiplication by the identity returns the other operator.

3.2.3 Tensor products of `tensopvar` objects

The function `otimes(A,B,type)` returns the tensor product $\mathcal{C}$ of $\mathcal{A}$ and $\mathcal{B}$ such that $\mathcal{C}(\mathbf{x}\otimes\mathbf{y}) = (\mathcal{A}\mathbf{x}) \odot (\mathcal{B}\mathbf{y})$ for `type(1)=1` (the default, `type` $= [1,2]$), and $(\mathcal{A}\mathbf{x}) \otimes (\mathcal{B}\mathbf{y})$ for `type(1)=2`, with a proper tensor product of the inputs. The operators are first expressed in terms of the same variables (`common_vars`). If $\mathcal{A}$ and $\mathcal{B}$ are defined by tensor products of the requested type, we just concatenate the factors: for each pair of terms, `ops(i,:)=[A.ops(iA,:),B.ops(iB,:)]`. Otherwise, $\mathcal{A}$ and $\mathcal{B}$ are stored as two nested factors `{A,B}` of the new operator, with `depmat` given by the maximum (type 1) or sum (type 2) of the degrees `deps` of $\mathcal{A}$ and $\mathcal{B}$. The function `kron(A,B)` is a different operation, representing the Kronecker product of the matrix-valued operators; it does not compute a tensor product of operators.

3.2.4 Application and integration of `tensopvar` objects

For $\mathbf{X} = \{x_1, \ldots, x_d\}$ a cell of `polynomial` objects, representing a distributed monomial $x_1 \otimes \cdots \otimes x_d$, `apply_op(Cop,X)` returns the function

$$\mathcal{R}(x_1 \otimes \cdots \otimes x_d) = \sum_{i=1}^{n_{\text{terms}}} (\mathcal{P}_{i,1}x_1) \circ \cdots \circ (\mathcal{P}_{i,d}x_d),$$

where $\circ$ is the elementwise product $\odot$ if `otype(1)=1`, and the Kronecker product $\otimes$ otherwise. In the latter case, the kth copy of an output variable s is renamed `s_k`. For nested operators, $\mathbf{X}$ is either a cell of the inputs to the nested operators, or has one input for each of the `num_factors` factors, in order.

The composition of the tensor-PI operator with an integral over its output domain is obtained using `int(Cop,W)`: this returns the `intop` object representing the functional

$$\mathcal{K}\mathbf{x} = \int_\Omega W(s)(\mathcal{R}\mathbf{x})(s)\, ds$$

for Ω the output domain of $\mathcal{R}$, and a weight W of type `double`, `polynomial` or `dpvar`, defaulting to the identity. A weight that depends on the output variables is only supported if each output variable has degree 1.

Further methods: `clean_op` (removes terms in which some factor is zero), `is_id_op`, `repmat`, `repelem`, `reorder_vars`, and concatenation (`horzcat`, `vertcat`, `blkdiag`, which return `tensopmat` objects). Indexing `C{i,j}` returns the operator in term i and factor j, and assigning to it re-checks the operator.

3.3 Concatenating Tensor-PI Operators: `tensopmat`

Using the `tensopvar` and `intop` classes, representing operators acting on distributed monomials, e.g. $\mathbf{x}^{\otimes d}$, we now develop the `tensopmat` class for concatenating such operators, in order to define coefficient operators acting on a full basis of distributed monomials, e.g. $(\mathbf{x}, \mathbf{x}^{\otimes 2}, \ldots, \mathbf{x}^{\otimes d})^T$. In particular, for different function spaces of the form

$$X_j = L_2^{n_j}[[a_1,b_1]^{k_{j,1}} \times \cdots \times [a_N,b_N]^{k_{j,N}}], \qquad Y_i = L_2^{m_i}[[c_1,d_1]^{\ell_{i,1}} \times \cdots \times [c_M,d_M]^{\ell_{i,M}}], \tag{4}$$

and given tensor-PI operators (or PI functionals) $\mathcal{R}_{i,j} : X_j \to Y_i$ for $i \in \{1,\dots,p\}$ and $j \in \{1,\dots,q\}$, we consider operators of the form

$$\mathcal{Q} = \begin{bmatrix} \mathcal{R}_{1,1} & \cdots & \mathcal{R}_{1,q} \\ \vdots & \ddots & \vdots \\ \mathcal{R}_{p,1} & \cdots & \mathcal{R}_{p,q} \end{bmatrix} : X_1 \times \cdots \times X_q \to Y_1 \times \cdots \times Y_p$$

We represent such operators as `tensopmat` objects, with the following fields.

tensopmat structure:

- `ops`: $p \times q$ cell with element (i,j) containing a `tensopvar` or `intop` object representing the operator $\mathcal{R}_{i,j}$. Elements can also be `dpvar`, `polynomial`, or `double` arrays, representing a multiplier operator acting on a degree-0 monomial ($X_j = \mathbb{R}^{n_j}$), see the Notes. Finally, an empty element is interpreted as a zero operator, where the input and output domains are determined from the other operators in the same row and column. Operators in the same column, j, must have the same input spaces, X_j, and operators in the same row, i, must have the same output spaces, Y_i.
- `dims`: struct with fields `in` and `out`. Field `in` is a $q \times 1$ array of integers, specifying for each column j of operators $\mathcal{R}_{i,j}$ the column dimension of those operators (they must all have the same column dimension). Field `out` is a $p \times 1$ arrays of integers, specifying for each row i of operators $\mathcal{R}_{i,j}$ the row dimensions of those operators.
- `vars`: struct with fields `in` and `out`, specified as $1 \times N$ and $1 \times M$ `cellstr` objects, respectively, specifying the names of the spatial variables on which the inputs and output functions to the operator depend.
- `dom`: struct with fields `in` and `out`, specified as $N \times 2$ and $M \times 2$ numeric arrays, respectively, specifying the intervals on which each of the input and output variables is defined.
- `depmat`: struct with fields `in` and `out`. Field `in` is a $q \times N$ array of integers, with element (j,r) specifying for column j of operators and input spatial variable r to what degree the operators $\mathcal{R}_{i,j}$ depend on this variable (value of $k_{j,r}$ in (4)). Similarly, field `out` is a $p \times M$ array of integers, with element (i,r) specifying for row i of operators and output spatial variable r to what degree the operators $\mathcal{R}_{i,j}$ depend on this variable (value of $\ell_{i,r}$ in (4)).
- `matdim`: 1×2 array of integers, $[m,n]$, specifying the total matrix dimensions of the operator: `matdim(1)=sum(dims.out)` and `matdim(2)=sum(dims.in)`.

As for the other classes, `vars`, `dom`, `dims` and `depmat` are determined from the blocks by `initialize`. A `tensopmat` is declared by `tensopmat(Cop)`, for a single operator or a cell of operators (also of type `sopvar`, `opvar` and `opvar2d`, which are converted to `tensopvar` objects), optionally with the variables and domains `tensopmat(Cop,vars,dom)` of the blocks of type `double`, `polynomial` and `dpvar`.

Notes:

- A block $\mathcal{R}_{i,j} = M$ of type `double`, `polynomial` or `dpvar` is an $m \times n$ matrix, representing the operator from $\mathbb{R}^n$ to $L_2^m[S]$ that maps x to the function Mx, where S are the spatial variables appearing in M (the decision variables of a `dpvar` are not spatial variables). Such a block can only act on $\mathbb{R}$, so column j has `depmat.in(j,:)=0`, and no other block in the column can act on a function space. A row consisting only of such blocks maps to the variables appearing in them; if the row has other blocks, these determine the output space, and the variables in M must be among them (a constant is a constant function). These blocks are used to represent coefficients acting on the constant monomial, e.g. a known function $m(s) = 1 - s^2$.
- The matrix structure of a `tensopmat` is such that the blocks are of size `dims.out(i)` by `dims.in(j)`; the elements of $\mathcal{Q}$ in the rows and columns of a block are not indexed separately. Indexing `C(i,j)` returns the `tensopmat` of the corresponding blocks, and `C{i,j}` the block itself.

3.3.1 Operations on `tensopmat` objects

The sum of two `tensopmat` objects (of the same block structure) is given by the sum of each of the blocks, where a block of type `double`, `polynomial` or `dpvar` can only be added to a block of such type, or to a `tensopvar` acting on a constant monomial. The block matrix product $\mathcal{AB}$ of $\mathcal{A} \in$ `tensopmat` with $\mathcal{B}$ (a `tensopmat`, `tensopvar`, `intop` or `sopvar`) is defined if the input space of $\mathcal{A}$ matches the output space of $\mathcal{B}$, as the usual block matrix product, where each block is a composition, see the multiplication of `tensopvar` and `intop` objects. Also supported are the product $M\mathcal{B}$ with a multiplier M of type `double` (a general matrix, if the output spaces of all rows of $\mathcal{B}$ match), or a scalar `polynomial` or `dpvar`, and the elementwise product $M.*\mathcal{B}$. The tensor product, `otimes(A,B,type)`, is defined blockwise, as for `tensopvar` and `intop`. For `tensopmat` objects $\mathcal{A} = [\mathcal{A}_{i,j}]$ and $\mathcal{B} = [\mathcal{B}_{i,j'}]$, with the default `type` $= [1, 2]$ (which requires matching row blocks), it returns the `tensopmat` $\mathcal{C}$ with block $(i, (j, j'))$ given by $\mathcal{A}_{i,j} \oslash \mathcal{B}_{i,j'}$, so that

$$\mathcal{C}(\mathbf{x}_j \otimes \mathbf{y}_{j'}) = (\mathcal{A}_{i,j}\mathbf{x}_j) \odot (\mathcal{B}_{i,j'}\mathbf{y}_{j'}),$$

i.e. $\mathcal{C}$ has a column block for each pair (j, j') of column blocks of $\mathcal{A}$ and $\mathcal{B}$ ($\mathcal{A}$'s outermost), acting on $X_j \otimes X'_{j'}$. For `type` $= [2, 2]$, block $((i, i'), (j, j'))$ is $\mathcal{A}_{i,j} \otimes \mathcal{B}_{i',j'}$. Block matrices can be concatenated by `horzcat`, `vertcat` and `blkdiag`, and repeated using `repmat` and `repelem`. Finally, `apply_op(Cop,X)` applies the operator to a cell $\mathbf{X}$ with one input for each column block, returning the vertical concatenation of the outputs of the p row blocks, $Y_i = \sum_j \mathcal{R}_{i,j} X_j$.

4 Representation of Distributed Polynomials: `polyopvar`

Having defined the class of tensor-PI operators acting on distributed monomials, we now define a class of distributed polynomials using such operators. Here, for given distributed state variables, $\mathbf{x}_k \in L_2^{n_k}[\Omega_k]$, $k \in \{1, \ldots, p\}$, where each state $\mathbf{x}_k$ depends on its own subset of the spatial variables, we consider a basis of distributed monomials of the form

$$Z(\mathbf{x}_1, \ldots, \mathbf{x}_p) = \begin{bmatrix} \mathbf{x}_{J(1,1)} \otimes \cdots \otimes \mathbf{x}_{J(1,d)} \\ \vdots \\ \mathbf{x}_{J(q,1)} \otimes \cdots \otimes \mathbf{x}_{J(q,d)} \end{bmatrix}, \tag{5}$$

where the entries of $J \in \{0, \ldots, p\}^{q \times d}$ index the states, and where a zero entry is not a state (see the definition of `Z` below). The degree of monomial i in state $\mathbf{x}_k$ is the number of entries $J(i, \cdot)$ equal to k. If $Z_i(\mathbf{x}_1, \ldots, \mathbf{x}_p) \in X_i$ with X_i of the form (4), we then define a distributed polynomial P in terms of the basis Z as

$$P(\mathbf{x}_1, \ldots, \mathbf{x}_p) = \mathcal{C}Z(\mathbf{x}_1, \ldots, \mathbf{x}_p) = \sum_{k=1}^{q} \mathcal{C}_{\cdot,k} Z_k(\mathbf{x}_1, \ldots, \mathbf{x}_p) \in Y_1 \times \cdots \times Y_r,$$

for a $r \times q$ block matrix $\mathcal{C} = [\mathcal{C}_{i,k}]$ of tensor-PI operators and PI functionals, $\mathcal{C}_{i,k} : X_k \to Y_i$, i.e. a `tensopmat`. Row i of P is then the polynomial $P_i = \sum_k \mathcal{C}_{i,k} Z_k$. Here, each of the function spaces Y_i may be $\mathbb{R}^{m_i}$ (in which case the operators $\mathcal{C}_{i,k}$ are functionals, `intop` objects, and P_i a distributed polynomial functional), or a space of functions in some of the spatial variables (the operators are `tensopvar` objects, and P_i a distributed polynomial function).

polyopvar structure:

- `C`: $m \times n$ `tensopmat` object, representing the coefficient operator defining the distributed polynomial, with block (i, k) acting on monomial k;
- `Z`: $q \times d_{\max}$ sparse array of integers J, defining the basis of monomials Z as in (5) in terms of which the polynomial is defined. Values of `Z` range from 0 to p, where p is the number of state variables, with `Z(i,j)=k` indicating that factor j in monomial i is given by $\mathbf{x}_k$. The degree of a monomial is the number of nonzeros in `Z(i,:)`. The zeros in `Z(i,:)` have two possible meanings:
  - if the tensor-PI operators acting on the monomial have at most as many factors as its degree, the zeros are just padding, and `initialize` moves them to the right. A monomial of degree 0, with `Z(i,:)` all zeros (e.g. `sparse(1,0)`), is the constant monomial $1 \in \mathbb{R}$, and the operators acting on it map from $\mathbb{R}$;

– if the operators have more factors, then factor j acts on state $\mathbf{x}_{\texttt{Z(i,j)}}$ if `Z(i,j)>0`, and is a multiplier from $\mathbb{R}$ (a known function, e.g. $m(s) = 1 - s^2$) to a function space, i.e. an operator with input dimension 1 and no input variables, if `Z(i,j)=0` (also for $j > d_{\max}$). For example, `Z(i,:)=[1,0,2]` represents $\mathbf{x}_1 \otimes m \otimes \mathbf{x}_2$, on which a tensor-PI operator with three factors with pointwise output, the identity, multiplier m, and the identity, acts to produce $\mathbf{x}_1(s)m(s)\mathbf{x}_2(s)$. The function `num_factors` returns the number of factors acting on each monomial, and `combine_factors` combines consecutive multipliers from $\mathbb{R}$, e.g. `Z(i,:)=[1,0,0,2]` to `[1,0,2]`, by multiplying them.

- `varname`: $p \times 1$ `cellstr`, specifying the names of the p distributed state variables on which the polynomial is defined, $(\mathbf{x}_1, \ldots, \mathbf{x}_p)$, stored in sorted order;
- `dims`: struct with fields `in`, a $p \times 1$ array specifying the vector sizes n_k of each of the state variables, and `out`, an $r \times 1$ array specifying for each row of `C` the row dimension of this row in the polynomial;
- `vars`: struct with fields `in`, a $1 \times N$ `cellstr` specifying the variable names in terms of which the state variables are defined, and `out`, a $1 \times M$ `cellstr` specifying the spatial variable names in terms of which the polynomial is defined (those of the output space of `C`);
- `dom`: struct with fields `in` and `out`, $N \times 2$ and $M \times 2$ `double` arrays specifying the intervals on which each of the input and output variables is defined;
- `depmat`: struct with fields `in`, a $p \times N$ `logical` array with element (k, r) indicating whether or not state variable $\mathbf{x}_k$ depends on spatial variable s_r, and `out`, an $r \times M$ array with element (i, j) indicating to what degree the ith row of the output depends on variable s_j.

Furthermore, `matdim` returns the matrix dimensions $[m, 1]$ of the polynomial, and `degmat` the $q \times p$ array of the degrees of the different state variables in each of the monomials, computed from `Z` (and not settable).

Declaring state variables: The state variables are declared by

```
polyopvar x1(s1,s2) x2(s1)
```

which declares the `polyopvar` objects `x1`, `x2` and the `pvar` objects `s1`, `s2` in the workspace (a variable `t` is assumed to be time, and is not declared), or by `x = polyopvar(varname,dims,vars,dom,depmat)`, where `dims` defaults to ones, `dom` to $[0, 1]$ for each variable, and `depmat` to all ones. Each state $\mathbf{x}_k$ is represented by the identity operator on the function space on which it is defined: `x.C` is a `tensopmat` with identity blocks on the diagonal, and `x.Z`$= k$, the monomial $\mathbf{x}_k$. Sums, products and other operations of such states return `polyopvar` objects, see the following.

Examples:

- For $\mathbf{x}(s_1, s_2)$ and $\mathbf{y}(s_1)$, the product `p=x.*y` has `p.varname`$=\{$`x,y`$\}$, `p.Z=[1,2]`, and `p.C` consists of a single `tensopvar` with two factors (the identity operators acting on $\mathbf{x}$ and $\mathbf{y}$), with `otype=[1,2]`: it maps $\mathbf{x}\otimes\mathbf{y} \in L_2[[0,1]^2 \times [0,1]]$, `deps.in=[2,1]`, to the function $(s_1, s_2) \mapsto x(s_1, s_2)y(s_1)$, `deps.out=[1,1]`.
- For two states $\mathbf{u}_1(s)$ and $\mathbf{u}_2(s)$, the functional $V = \langle \mathbf{u}_1, \mathbf{u}_1 \rangle + \langle \mathbf{u}_2, \mathbf{u}_2 \rangle$, `V=innerprod([u1;u2],[u1;u2])`, has `V.Z=[1,1;2,2]`, with `C` a 1×2 `tensopmat` of two `intop` objects, each with `intmat=`$\{$`[0,3,1,1]`$\}$ and acting on $L_2[[0,1]^2]$. Note that the monomials $\mathbf{u}_1 \otimes \mathbf{u}_2$ do not appear, as these have coefficient zero.
- The sum `f=u1+u1.*u2` has `f.Z=[1,0;1,2]`: the first monomial is of degree 1, and the zero is padding.

Notes

- Since $\mathbf{x} \otimes \mathbf{y} \neq \mathbf{y} \otimes \mathbf{x}$ for distributed $\mathbf{x} \neq \mathbf{y}$, in general, the order of the factors in the distributed monomials matters. Therefore, `Z` will be the primary field for representing distributed monomials. The field `degmat` still provides the degrees of different state variables in each monomial, but different monomials may have the same degrees. Two monomials with the same `Z` but a different number of factors (of the operators acting on them, see above) are different monomials.

- The `in` fields for e.g. `depmat` and `dims` are associated with the state variables, not the monomials, defining the function space on which the distributed polynomial acts. As such, whereas a well-defined `polyopvar` structure must have `depmat.out=C.depmat.out` (and likewise for `vars.out`, `dom.out`, `dims.out`), it will not satisfy `depmat.in=C.depmat.in`, in general: the input space of `C` is that of the monomials, `vars.in` may differ from `C.vars.in` (a variable of the polynomial that `C` lacks must not appear in any monomial, and a variable of `C` that the polynomial lacks must not be acted on).
- At this time, matrix-valued polynomials are not supported. Vector-valued polynomials are supported.
- The fields `C`, `Z`, ... can be set outside the methods of the class, e.g. `f.C = Cnew`. The object is then marked as not checked (`chkval=false`), and `initialize` must be called to check the consistency of the fields and to fill in `dims`, `vars`, `dom` and `depmat` (inferred from `C` if left empty).

4.1 Addition of `polyopvar` objects

Express the polynomials in terms of the same basis of monomials using `common_basis`, and then take the sum of the coefficients. In particular, the polynomials are first expressed in terms of the same state variables and spatial variables (`common_states`, `common_vars`), after which the monomials, `Z`, are combined into a unique list, and the coefficient operators are extended with zero blocks for the monomials that do not appear in the polynomial. The sum is then $P_A + P_B = (\mathcal{C}_A + \mathcal{C}_B)Z$. Terms acting on the same monomial are added (`combine_terms`); if the polynomials have no output spatial variables (functionals), monomials of the same degree in each state, in a different order, are combined as well, by reordering the integrals defining the operators acting on them, e.g. $\int_a^b \int_c^d y(s_1)x(s_2)ds_2ds_1 = \int_c^d \int_a^b x(s_1)y(s_2)ds_1ds_2$. Addition of polynomials depending on different spatial variables is not supported: the output spaces must match, e.g. $p_1(\mathbf{x})(s_1) + p_2(\mathbf{y})(s_2)$ is not defined, as we would have to augment $p_1(\mathbf{x})(s_1, s_2)$ and $p_2(\mathbf{y})(s_1, s_2)$. A constant of type `double`, `polynomial` or `dpvar` can be added to a polynomial with matching output space, by representing it as a degree-0 monomial (`constant2polyopvar`).

4.2 Multiplication of `polyopvar` objects

Multiplication is assumed to be pointwise in space, but not elementwise. Pointwise multiplication is performed by taking the tensor product of the bases (type 2), and the type-1 tensor product of the operators. Specifically, for $P_A = \sum_i \mathcal{C}_i^A Z_i^A$ and $P_B = \sum_k \mathcal{C}_k^B Z_k^B$, the product is

$$P_A \odot P_B = \sum_{i,k} (\mathcal{C}_i^A \oslash \mathcal{C}_k^B)(Z_i^A \otimes Z_k^B),$$

so that the monomials of the product are the products of each monomial of P_A with each monomial of P_B, where the factors of P_B follow all those of P_A (including multipliers from $\mathbb{R}$, i.e. zeros in `Z`): `Z` = `[A.Z(i,:),B.Z(k,:)]`, and where the coefficient operators are given by `otimes(A.C,B.C,[1,2])`. The result is simplified by combining terms involving the same monomial. The product `A.*B` is defined if the dimensions match elementwise (a scalar is repeated), and `A*B` is defined if one of the polynomials is scalar. Multiplication of a polynomial by a constant is performed on the coefficient operators, `C.*A`, i.e. $\mathcal{C} \mapsto M\mathcal{C}$.

4.3 Other operations on `polyopvar` objects

- `vertcat`: concatenation of polynomials in a column, after expressing them in terms of the same basis; the input spaces of the coefficients and the output domains must match. Horizontal concatenation, `horzcat`, is not supported, as it would produce a matrix-valued polynomial.
- `subs(f,var,val)`: substitution of the state variables `var` (specified by name, or the factors by number) in the monomials of f by a distributed polynomial or a constant `val`. For `var=`$\mathbf{x}_1$ and $f(\mathbf{x}) = \mathcal{K}(\mathbf{x}_1^{\otimes d_1} \otimes \cdots)$, each factor $\mathbf{x}_1$ is replaced by $p(\mathbf{x})$, so that f is expressed in terms of the states of p. The states are replaced one at a time. If `val` is a constant, or only some of the factors in a monomial are replaced, the remaining factors become known factors (multipliers from $\mathbb{R}$, zeros in `Z`). This is used e.g. to express a functional in the PDE state $\mathbf{u}$ in terms of the PIE state $\mathbf{x}$, `subs(V,u,Top*x)`, using $\mathbf{u} = \mathcal{T}\mathbf{x}$.
- `apply_poly(P,xvals)`: evaluates the polynomial at given values (`polynomial` objects) of the state variables, returning $P(\mathbf{x}) = \mathcal{C}Z(\mathbf{x})$ by applying the operators to the monomials.
- `clean_poly`: removes coefficients below a tolerance, merges duplicate monomials and consecutive known factors (`combine_factors`), and removes monomials and state variables that do not contribute (`remove_unused_states`).

- `ispvar`: checks whether the object is an array of individual state variables (identity operator acting on a state); indexing `P(i,j)` returns the `polyopvar` of the selected rows, and `P{i,j}` the term of row i acting on monomial j; `repmat`, `repelem`, `uminus`, `minus`, `size`.

4.4 Inner products and monomial bases

Distributed polynomial functionals are obtained through inner products of distributed polynomial functions. For $\mathbf{u}_1, \mathbf{u}_2$ vectors of `polyopvar` objects, and a weight $P(s)$ of type `double`, `polynomial` or `dpvar` (or a cell array of blocks, partitioned as the vectors), `V=innerprod(u1,P,u2)` returns the scalar `polyopvar` representing

$$V = \langle \mathbf{u}_1, P\mathbf{u}_2 \rangle = \int_\Omega \mathbf{u}_1(s)^T P(s) \mathbf{u}_2(s)\, ds,$$

with $P = 1$ and $\mathbf{u}_2 = \mathbf{u}_1$ by default. For $\mathbf{u}_1 = \sum_i \mathcal{C}_i^1 Z_i^1$ and $\mathbf{u}_2 = \sum_k \mathcal{C}_k^2 Z_k^2$, the monomials of V are the products $Z_i^1 \otimes Z_k^2$, with coefficients the functionals obtained by integrating the pointwise product of the operators, `int(otimes(C1,C2,[1,2]),P)` (see `int` for `tensopvar` objects). If the weight is symmetric, only half of the off-diagonal blocks are computed and doubled.

A vector of distributed monomials in given states is declared by `Z=dmonomials(vartab,degs)`, with `degs` an $n \times 1$ array of cumulative degrees, returning all monomials up to those degrees, or an $n \times p$ array specifying the degree of each monomial in each of the p states. The rows of `vartab` may also be polynomials in the states, e.g. `Top*y`.

5 SOS Programming with Distributed Polynomials

The functions in this section use the classes above to pose optimization problems with conditions that must hold for all distributed states $\mathbf{x} \in L_2^n[\Omega]$. The program is declared in the SOSTOOLS format, with the spatial variables and the dummy variables as independent variables, and the decision variables of the operators as decision variables.

Declaring a program. `prog = piesos_program(vartab,decvartab)` declares a program structure, where `vartab` is a vector of `polyopvar` state variables, and `decvartab` optionally a vector of `dpvar` decision variables. The program is solved using `prog = piesos_solve(prog,opts)`, an objective to minimize is set by `piesos_setobj(prog,obj)`, and the value of a decision variable (`polyopvar`, `tensopmat`, `tensopvar`, `intop` or `dpvar`) in the solution is obtained by `piesos_getsol(prog,dvar)`.

Declaring decision variables. Polynomial functionals with unknown coefficients are declared in the following ways.

- `piesos_intopvar(prog,matdim,vars,dom,deps,intmat,degs)` declares a functional variable $\mathcal{K}$, an `intop` object whose kernels have decision variables as coefficients, with the integrals given by `intmat` (a cell, or one of `'full'`, `'nomult'`, `'lower'`, `'upper'`) and the degrees of the monomials in the kernels by `degs`.
- `piesos_polyvar(prog,Zx,opts)` declares an indefinite distributed polynomial functional $p = \mathcal{C} Z_x$, in the basis of distributed monomials Z_x, with $\mathcal{C} = [\mathcal{K}_1, \ldots, \mathcal{K}_q]$ and $\mathcal{K}_i$ a functional variable (`piesos_intopvar`) acting on monomial i.
- `piesos_quadvar(prog,ZL,ZR,opts)` declares the quadratic functional

$$V(\mathbf{x}) = \langle \mathcal{Z}_L Z_L(\mathbf{x}), P(s)\, \mathcal{Z}_R Z_R(\mathbf{x}) \rangle \tag{6}$$

  for bases of distributed monomials Z_L, Z_R, and bases of tensor-PI operators $\mathcal{Z}_L, \mathcal{Z}_R$ (`tensopmat` objects, one block for each monomial, acting on the monomials in Z_L, Z_R). Each operator is a sum of multiplier terms and lower and upper integral terms (3-PI operators, see `monopmat`) with kernels given by monomials of a specified degree d in the dummy variables, and maps the monomial to a function of the primary variables s. The matrix $P(s) = Z_s(s)^T Q Z_s(s)$ is a polynomial matrix in s, with Q a decision-variable matrix: general by default, symmetric (`opts.sym`), or positive semidefinite (`opts.pos`), in which case $P(s) \succeq 0$ for all s (an SOS matrix). Options (`degs`, `exclude`, `sep`, `psatz`) specify the degrees d (`degs.in`) and p (`degs.out`, the degree of Z_s), which of the multiplier (`exclude(1)`), lower (`exclude(2)`) and upper (`exclude(3)`) integral terms to exclude, whether to enforce separable kernels, and psatz multipliers: if `psatz`$(i) = q_i$, then $P(s) = \sum_k g_k(s) P_k(s)$, with P_k SOS and $g_k(s) = \prod_i (b_i - s_i)^{q_i - j_i} (s_i - a_i)^{j_i}$, $j_i \in \{0, \ldots, q_i\}$, which is nonnegative on the domain.

- `piesos_sosvar(prog,Z,opts)` declares the distributed SOS polynomial $V = \langle \mathcal{Z}Z(\mathbf{x}), P(s)\mathcal{Z}Z(\mathbf{x})\rangle$ in (6) with $Z_L = Z_R = Z$, $\mathcal{Z}_L = \mathcal{Z}_R = \mathcal{Z}$ and $P(s) \succeq 0$. Since

$$V(\mathbf{x}) = \int_\Omega \big(\mathcal{Z}Z(\mathbf{x})\big)(s)^T P(s) \big(\mathcal{Z}Z(\mathbf{x})\big)(s)\, ds \geq 0$$

  for all $\mathbf{x}$ when $P(s) \succeq 0$ for all $s \in \Omega$, V is nonnegative. States in a monomial of Z that depend on different spatial variables cannot be combined in the basis.

- `piesos_lpivar`, `piesos_poslpivar` and `piesos_quadopvar` declare the corresponding operator variables on function spaces, $\mathcal{P} = P_{\text{mat}}[\mathcal{Z}_1; \ldots; \mathcal{Z}_q]$ (indefinite), and the positive operator $\mathcal{P} = \mathcal{Z}^* P \mathcal{Z}$, represented only by P and $\mathcal{Z}$, since the operator itself cannot be represented by a `tensopmat`.

Equality and inequality constraints. `piesos_eq(prog,F)` imposes $F(\mathbf{x}) = 0$ for all $\mathbf{x}$ by requiring that the coefficient operator of each monomial of F is zero: F is first cleaned (`clean_poly`), combining terms acting on the same monomial and bringing the functionals to the canonical form of `combine_terms`, after which the kernels of each coefficient functional are required to vanish identically, using `soseq` for each kernel (a polynomial in the dummy variables, with coefficients that are linear in the decision variables).

`piesos_ineq(prog,F,opts)` imposes $F(\mathbf{x}) \geq 0$ for a scalar distributed polynomial functional F (coefficients of type `intop`) by declaring a distributed SOS polynomial $G = \langle \mathcal{Z}Z, Q\mathcal{Z}Z\rangle$ using `piesos_sosvar`, and imposing $F = G$ using `piesos_eq`. The basis Z consists of the distributed monomials with degrees between half the minimal and half the maximal total degree of F, inside the Newton polytope of the degrees of F halved (monomials combining states that depend on different spatial variables are discarded). The degrees d and p of $\mathcal{Z}$ and Q, and the types of terms in $\mathcal{Z}$, are chosen based on the exponents of the monomials of the dummy variables in the kernels of F (see the documentation of `piesos_ineq`); they can be overridden in `opts.degs` and `opts.exclude`, and psatz multipliers specified in `opts.psatz`. This is a sufficient condition, and conservative if the chosen degrees are too small, or if monomials combining states that depend on different spatial variables are required (these are discarded with a warning).

Lyapunov functionals and Lie derivatives. A PDE is converted to a PIE (possibly nonlinear),

$$\frac{d}{dt}\big(\mathcal{T}\mathbf{x}(t)\big) = f(\mathbf{x}(t)),$$

by `PIE = convert_PIETOOLS_PDE_nonlinear(PDE)`, with $\mathcal{T}$ a `tensopmat` (`PIE.T`) mapping the PIE state $\mathbf{x}$ (`PIE.x`) to the PDE state $\mathbf{u} = \mathcal{T}\mathbf{x}$, and f a `polyopvar` (`PIE.f`). The call

```
[prog,V,dV,Pmat,Zop] = piesos_LF(prog,PIE,xvars,degmat,opts)
```

declares a distributed polynomial Lyapunov functional in the PDE states,

$$V(\mathbf{x}) = \langle \mathcal{Z}Z(\mathbf{u}), P(s)\mathcal{Z}Z(\mathbf{u})\rangle\,, \qquad \mathbf{u} = \mathcal{T}\mathbf{x},$$

as in (6) (positive, if `opts.pos`), for the monomial basis Z specified by `degmat`, or in terms of the PIE states, and computes its Lie derivative along the PIE using `Liediff`. For a monomial $V(\mathbf{u}) = \mathcal{K}(\mathbf{u}_{i_1} \otimes \cdots \otimes \mathbf{u}_{i_d})$, the product rule gives

$$dV(\mathbf{x}) = \sum_{j=1}^{d} \mathcal{K}\big(\mathbf{u}_{i_1} \otimes \cdots \otimes f_{i_j}(\mathbf{x}) \otimes \cdots \otimes \mathbf{u}_{i_d}\big),$$

with the remaining factors $\mathbf{u}_{i_k}$ replaced by $\mathcal{T}\mathbf{x}$, so that dV is a `polyopvar` in the PIE states. For P symmetric, this is $dV(\mathbf{x}) = 2\left\langle \mathcal{Z}Z(\mathbf{u}), P\,\mathcal{Z}\frac{d}{dt}Z(\mathbf{u})\right\rangle$. Stability can then be certified by imposing $-dV \geq 0$ using `piesos_ineq`. The inner product $\langle \mathbf{u}, \mathbf{u}\rangle$ is obtained from `innerprod(u,u)`, with $\mathbf{u} = \mathcal{T}\mathbf{x}$ a `polyopvar`, and its derivative as `2*innerprod(u,f)`.

Example. Stability of the Burgers equation, $u_t = u_{ss} + ru - uu_s$, $u(0) = u(1) = 0$, is tested (see `Examples/PIESOS_Burgers.m`) by

```
PIE  = convert_PIETOOLS_PDE_nonlinear(PDE,'silent');   % PIE: T*x_t = f(x)
Top = PIE.T;    f = PIE.f;    x = PIE.x;
prog = piesos_program(x);                          % PIESOS program in PIE state x
P_opts.degs = 3;  P_opts.sep = 1;  P_opts.pos = true;
d = 1;                                             % degree of the monomials Z(u)
[prog,Vx,dV,Pmat,Zop] = piesos_LF(prog,PIE,PIE.varname_pde,(1:d)',P_opts);
```

```
u = Top*x;                                          % add eps*||u||^2 to V
Vx = Vx + eppos*innerprod(u,u);   dV = dV + 2*eppos*innerprod(u,f);
prog = piesos_ineq(prog,-dV,struct('psatz',1)); % enforce dV <= 0
prog_sol = piesos_solve(prog);
```

declaring a distributed SOS Lyapunov functional V in the PDE state, computing its derivative dV along the PIE, and enforcing $-dV \geq 0$ by an SOS constraint.